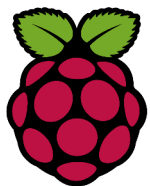


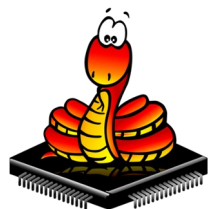


Atelier N°15 La Mini Serre Le servo moteur

Atelier Raspi

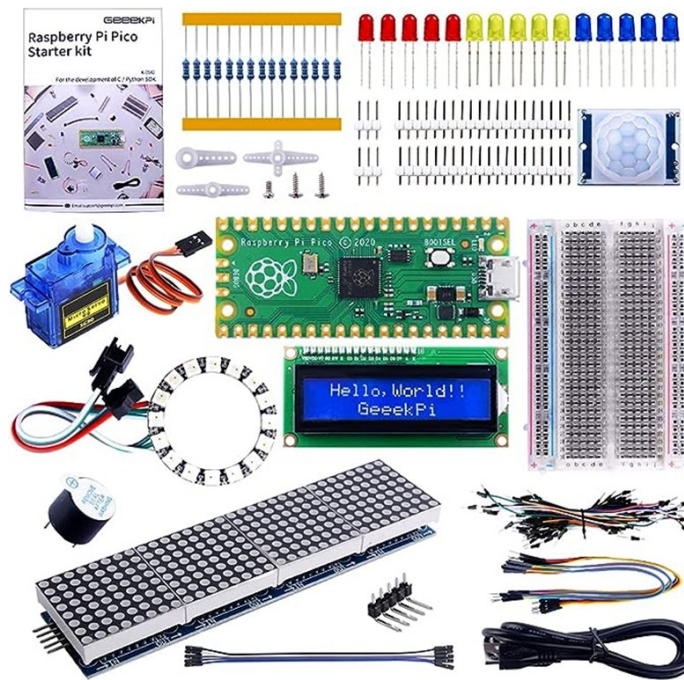


Logo du Raspberry Pico



Logo du MicroPython

L'atelier a pour valeurs, le partage, l'aide,
la formation, le faire et construire
ensemble à partir de l'expérience des
participants



Atelier N°15 La Mini Serre

Le servo moteur

Etape 1 Le champ magnétique

Etape 2 Les servo moteurs

Etape 3 L'asservissement

Etape 4 Le signal de commande (1/2)

Etape 5 Le signal de commande (2/2)

Etape 6 Les fonctions du module PWM

Etape 7 Le montage

Etape 8 Bibliothèque de code : Les tests

Etape 9 Bibliothèque de code : Les codes plus complexes

Etape 10 Le programme de test du servo moteur

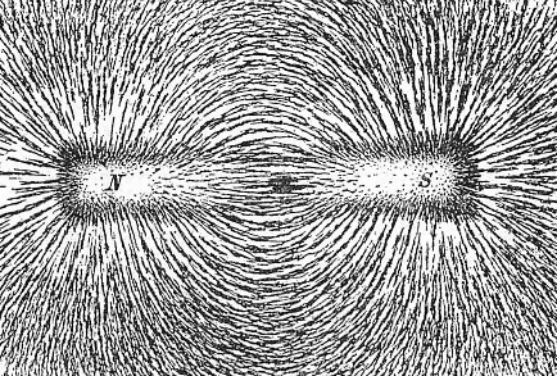
Etape 11 Servo Moteur et Bouton Poussoir

Etape 12 Servo Moteur et 2 Boutons Poussoirs

Etape 13 Application 1

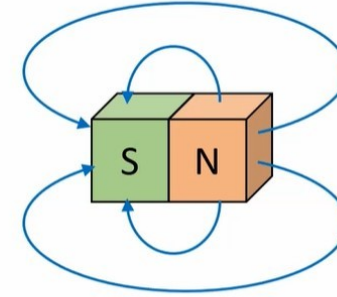
Etape 14 Application 2

A15E1 Le champ magnétique

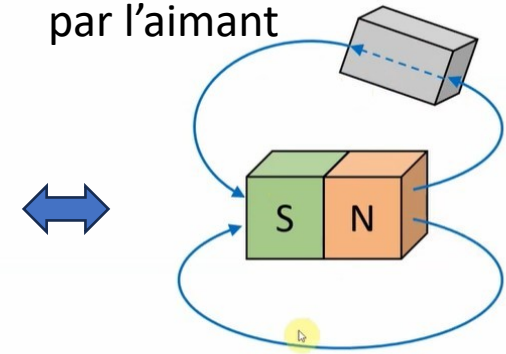


Superposition de:
limaille de fer,
papier,
aimant

Visualisation du champ magnétique crée par l'aimant

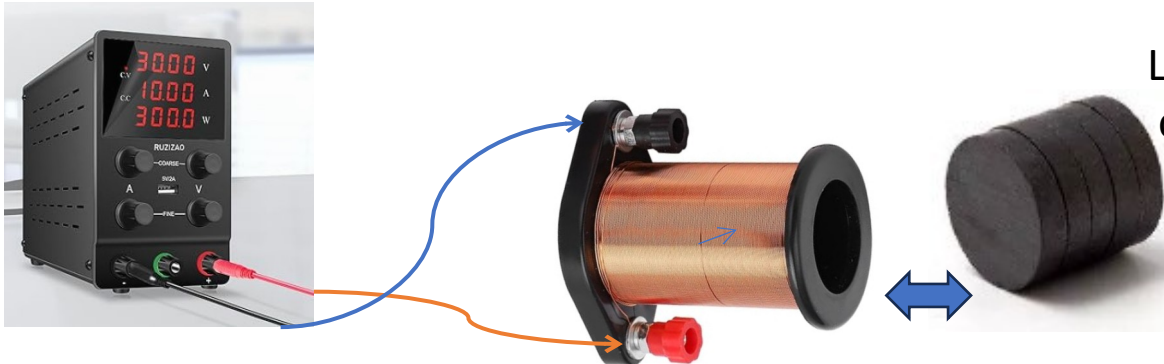


Des lignes de champ forme un spectre magnétique autour de l'aimant.
Elles se déplacent du nord au sud à l'extérieur de l'aimant.
Elles sortent et entrent à angle droit.

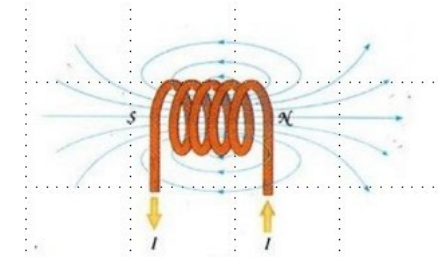


Le fer doux guide les lignes du champs magnétique et est attiré par l'aimant

Un matériaux magnétique placé dans le champ magnétique va dévier les lignes de champ. Elles prennent le trajet qui présente le moins d'opposition à leur passage (réductance).



L'aimant est attiré
ou repoussé par la bobine



Visualisation du champ magnétique
créé par la bobine

C'est la circulation du courant dans la bobine qui crée le champ magnétique

Servo-moteurs

- Un **servomoteur** (vient du latin *servus* qui signifie « esclave ») est un moteur capable de maintenir une opposition à un effort statique et dont la position est vérifiée en continu et corrigée en fonction de la mesure. C'est donc un système **asservi**.

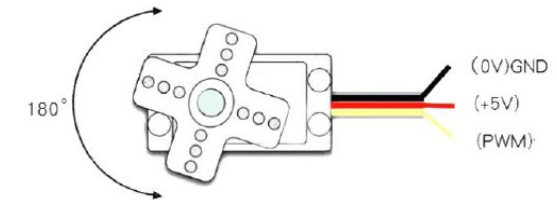
C'est un ensemble mécanique et électronique comprenant :

- un moteur à courant continu
- un réducteur en sortie de ce moteur diminuant la vitesse mais augmentant le couple ;
- un potentiomètre (faisant fonction de diviseur résistif) qui génère une tension variable, proportionnelle à l'angle de l'axe de sortie ;
- un dispositif électronique d'asservissement ;
- un axe dépassant hors du boîtier avec différents bras ou roues de fixation.

Les servos peuvent actionner les parties mobiles d'un robot, drone, etc

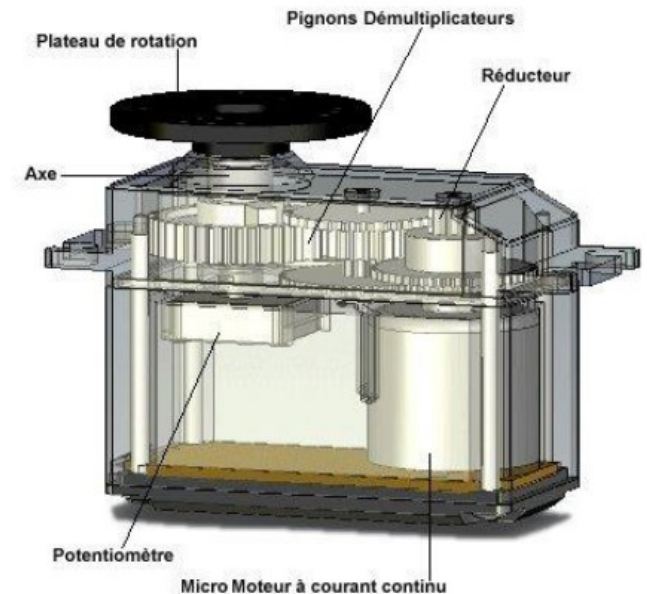
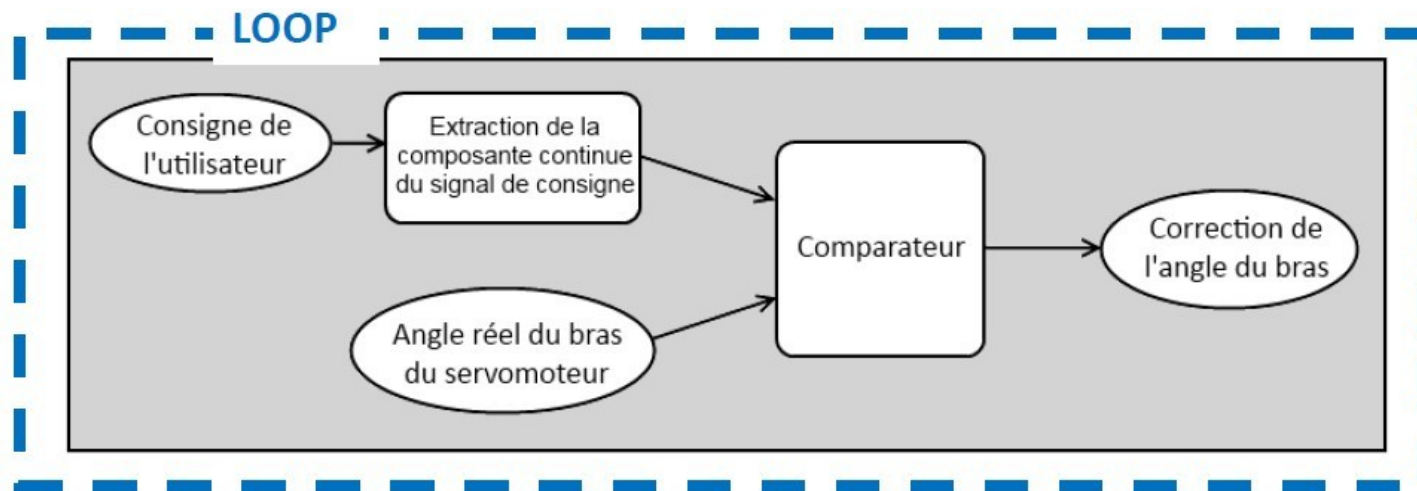


interface definition



Servo-moteurs : Asservissement ?

- C'est un moyen de gérer corriger une commande en fonction d'une consigne et d'un capteur de position
- Par exemple pour un servomoteur : on lui envoie un signal de commande qui définit l'angle désiré (consigne), et le moteur va corriger sans angle de départ pour tendre vers la consigne de l'utilisateur. Et il réalise cette opération en boucle.
- Pour pouvoir réaliser la correction de l'angle du bras, le servo utilise une électronique d'asservissement. Cette électronique est constituée d'un comparateur qui compare la position du bras du servo à la consigne.
- La position du bras est obtenue grâce à un potentiomètre couplé à l'axe du moteur.
- Après une rapide comparaison entre la consigne et valeur réelle de position du bras, le servomoteur (du moins son électronique de commande) va appliquer une correction si le bras n'est pas orienté à l'angle imposé par la consigne.

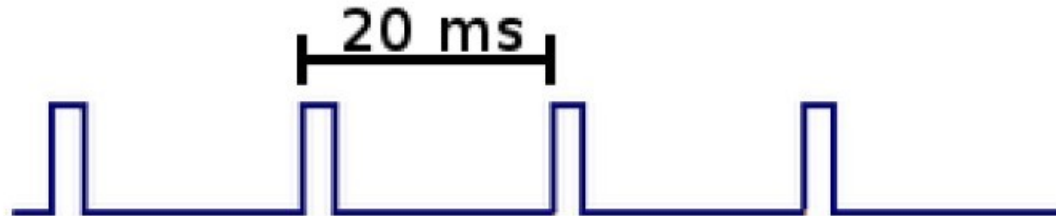


Servo-moteurs : Signal de commande (1/2)

La consigne envoyée au servomoteur est un signal électronique de type PWM. Il dispose cependant de deux caractéristiques indispensables pour que le servo puisse fonctionner : la fréquence fixe et la durée de l'état haut

LA FRÉQUENCE FIXE

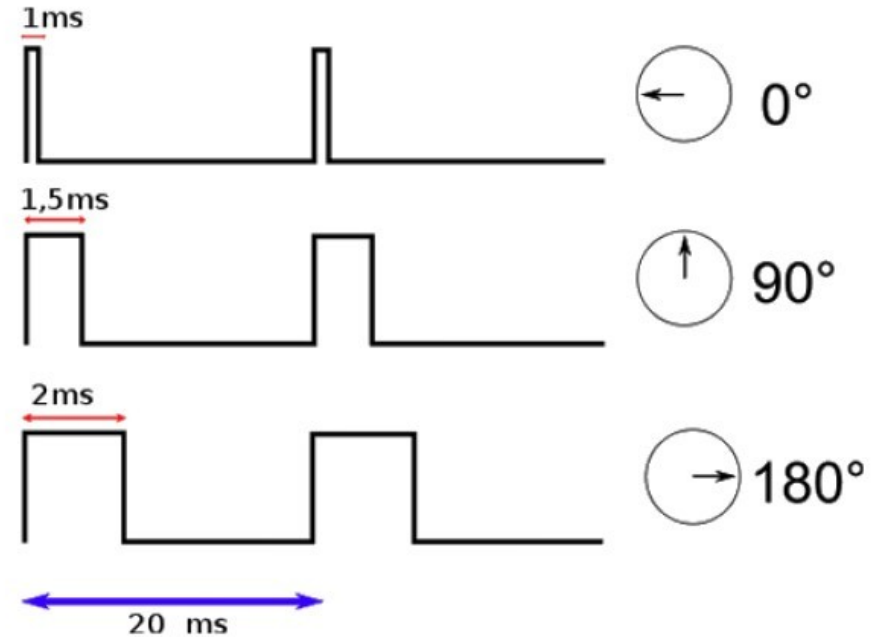
- Le signal que nous allons devoir générer doit avoir une fréquence de 50 Hz. Autrement dit, le temps séparant deux fronts montants est de 20 ms.
- La fonction `analogWrite()` de Arduino ne pourra donc pas utiliser cette fonction car on ne peut pas régler la fréquence



Servo-moteurs : Signal de commande (2/2)

LA DURÉE DE L'ÉTAT HAUT

- Cette durée indique au servomoteur l'angle précis qui est souhaité par l'utilisateur.
- Un signal ayant une durée d'état HAUT de 1ms donnera un angle à 0° , le même signal avec une durée d'état HAUT de 2ms donnera un angle au maximum de ce que peut admettre le servomoteur.



A15E6 Les fonctions du module PWM

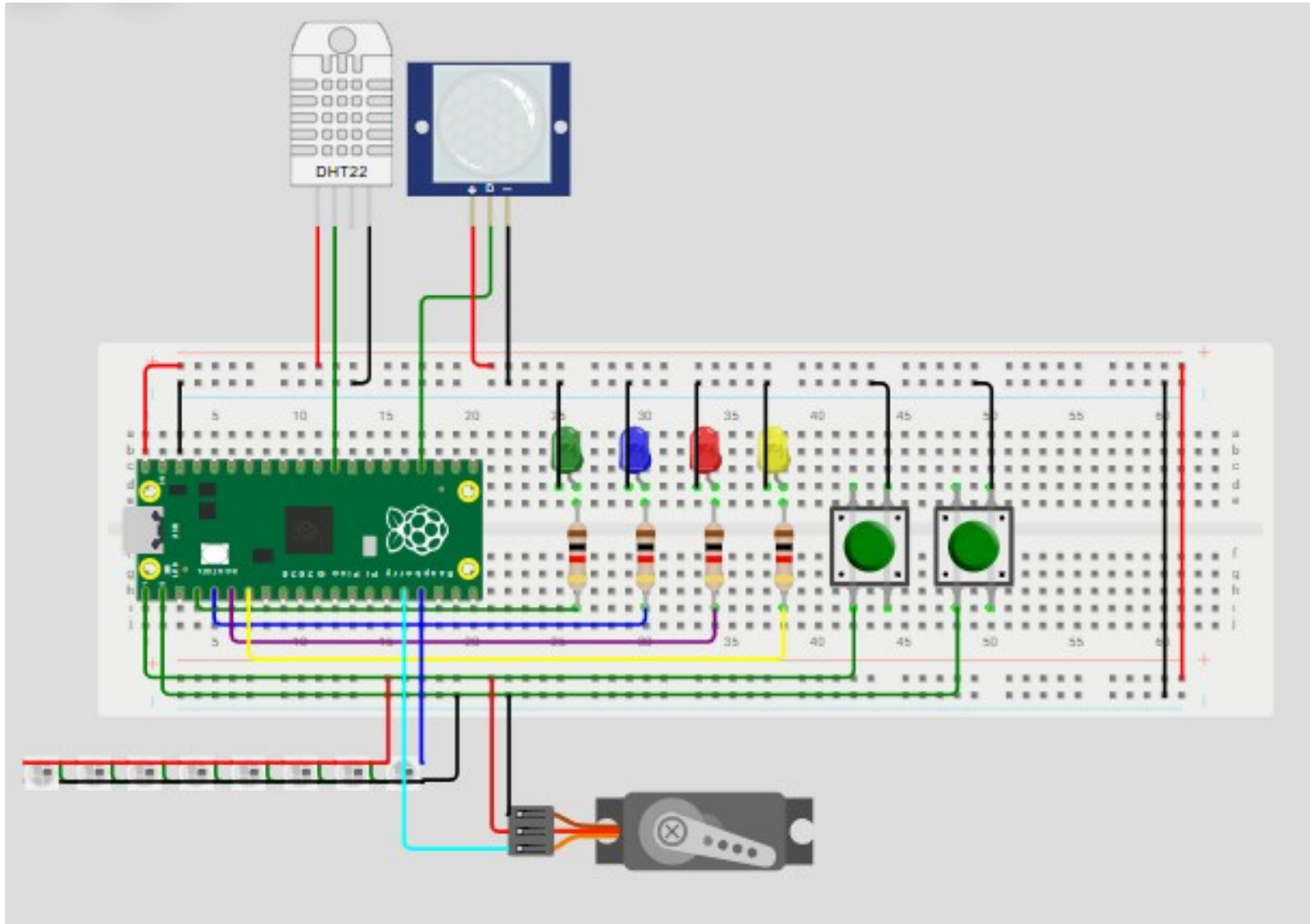
Fonctions utilisées:

```
servo = PWM(Pin(12))      # servo est un objet, initialise le Pin GPIO 12 en sortie PWM  
servo.freq(50)            # Initialise la fréquence des signaux carrés à 50Hz soit 20ms  
servo.duty_u16(int(newDuty)) # duty_u16 est le rapport cyclique entre le temps du signal carré haut sur sa  
période, new duty est une variable, le max est à 65535.
```

```
servo.deinit()           # réinitialise le PWM
```

```
servoX.duty_ns(pulseDX) #commande le rapport cyclique du PWM en nanosecondes suivant la variable  
pulseDX de l'objet servoX
```


A15E7 Le Montage

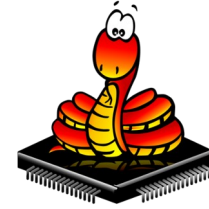


Composant	GPIO
Led Verte	2
Led Bleue	3
Led Rouge	4
Led Jaune	5
DHT11/ DHT22	22
PIR	18
Ruban Neopixel	13
BP1	0
BP2	1
Servo	12

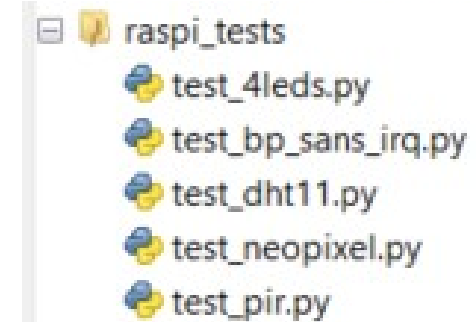
Code couleur des câbles :

- Vcc : Rouge / Orange
- Gnd : Noir / Marron
- Données : Vert / Bleu / Jaune / Violet

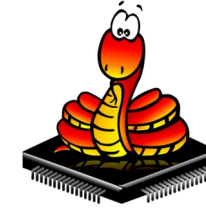
A15E8 Bibliothèque de code : les tests



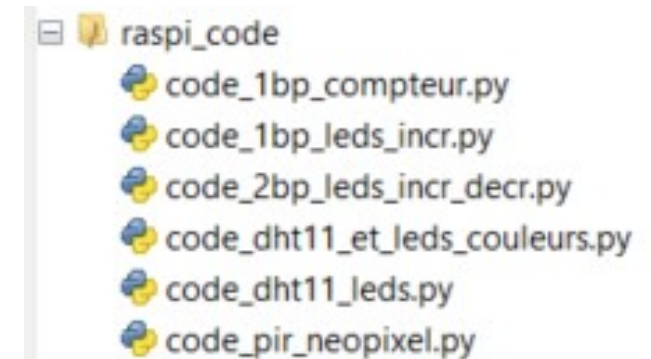
Tests	Résultat attendu
test_4leds	Les leds s'allument les unes après les autres
test_dht11	Affiche la température dans la console
test_pir	Affiche un message dans la console
test_neopixel	Allume le ruban de différentes couleurs
test_bp_sans_irq	Indique l'état du bouton dans la console : appuyé ou relaché



A15E9 Bibliothèque de code : les codes plus complexes



Code	Résultat attendu
code_dht11_leds	Affiche la température sur les leds
code_dht11_et_leds_couleur	Affiche la température en utilisant la couleur des leds
code_pir_neopixel	Allume le ruban quand une présence est détectée
code_1bp_compteur	Incrémente un compteur à chaque appui sur le bouton
code_1bp_leds_incr	Allume une led supplémentaire à chaque appui sur BP1
code_2bp_leds_incr_decr	Allume une led à chaque appui sur BP1, éteint une led à chaque appui sur BP2



A15E10 Le programme de test du Servo Moteur

```
1 from machine import Pin, PWM
2 from time import sleep
3
4 servo = PWM(Pin(12)) # fil jaune sur GPIO 12
5 servo.freq(50)
6
7 while True:
8     servo.duty_u16(2000) # gauche - angle 0 degrés
9     sleep(1)
10
11     servo.duty_u16(5000) # milieu - angle 90 degrés
12     sleep(1)
13
14     servo.duty_u16(8000) # droite - angle 180 degrés
15     sleep(1)
```

✅ 50 Hz = le bon rythme

Les servos classiques (SG90, MG90...) sont faits pour :

❤️ 1 message toutes les 20 ms

Donc :

python

servo.freq(50)

0°	-----	90°	-----	180°
2000		5000		8000

Quand on écrit :

python

servo.duty_u16(5000)

👉 on demande au servo d'aller à 90 degrés.

1234 Pourquoi 5000 = 90° ?

Parce que :

- 2000 ≈ 0°
- 8000 ≈ 180°

Le milieu, c'est :

yaml

(2000 + 8000) ÷ 2 = 5000

A15E11 Servo Moteur et Bouton Poussoir

Chaque appui sur le bouton poussoir 1 augmente l'angle du servo moteur

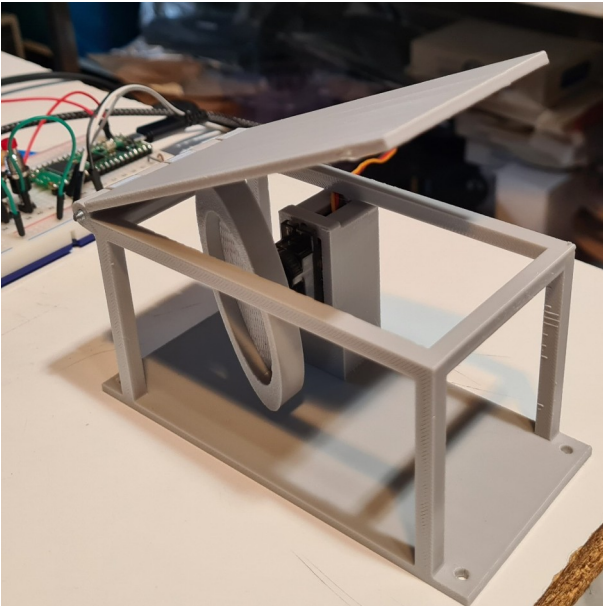
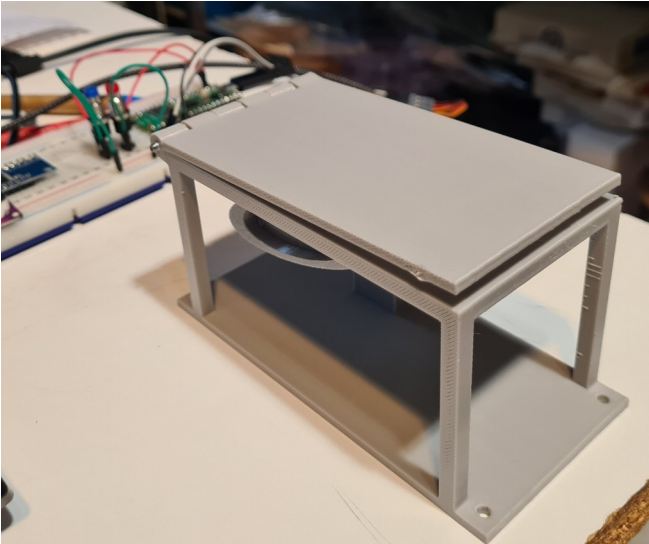
```
1 from machine import Pin, PWM
2 import time
3
4 #Declaration du bouton
5 bouton = Pin(0, machine.Pin.IN, Pin.PULL_UP)
6
7 #Declaration du servo
8 servo = PWM(Pin(12))
9 servo.freq(50)
10 servo.duty_u16(2000)
11
12 #Declaration d'une variable compteur
13 compteur = 2000
14
15 while True:
16     #On regarde l'etat du bouton
17     if bouton.value() == 0:
18         compteur = compteur + 1000
19         print("Compteur = ",compteur)
20
21     #On verifie et on corrige si besoin
22     #les valeurs du compteur
23     if (compteur > 8000):
24         compteur = 8000
25
26     #On envoie la valeur au servo
27     servo.duty_u16(compteur)
28
29     #On attends un peu (pour éviter les rebonds du BP)
30     time.sleep(0.3)
31
```

A15E12 Servo Moteur et 2 Boutons Poussoirs

Chaque appui sur le bouton poussoir 1
augmente l'angle du servo moteur
Chaque appui sur le bouton poussoir 2
diminue l'angle du servo moteur

```
1 from machine import Pin, PWM
2 import time
3
4 #Declaration du bouton
5 bouton = Pin(0, machine.Pin.IN, Pin.PULL_UP)
6 bouton2 = Pin(1, machine.Pin.IN, Pin.PULL_UP)
7
8 #Declaration du servo
9 servo = PWM(Pin(12))
10 servo.freq(50)
11 servo.duty_u16(2000)
12
13 #Declaration d'une variable compteur
14 compteur = 2000
15
16 while True:
17     #On regarde l'etat du bouton 1
18     if bouton.value() == 0:
19         compteur = compteur + 1000
20         print("Compteur = ",compteur)
21
22     #On regarde l'etat du bouton 2
23     if bouton2.value() == 0:
24         compteur = compteur - 1000
25         print("Compteur = ",compteur)
26
27     #On verifie et on corrige si besoin
28     #les valeurs du compteur
29     if (compteur > 8000):
30         compteur = 8000
31     if (compteur < 2000):
32         compteur = 2000
33
34     #On envoie la valeur au servo
35     servo.duty_u16(compteur)
36
37     #On attends un peu (pour éviter les rebonds du BP)
38     time.sleep(0.3)
39
```

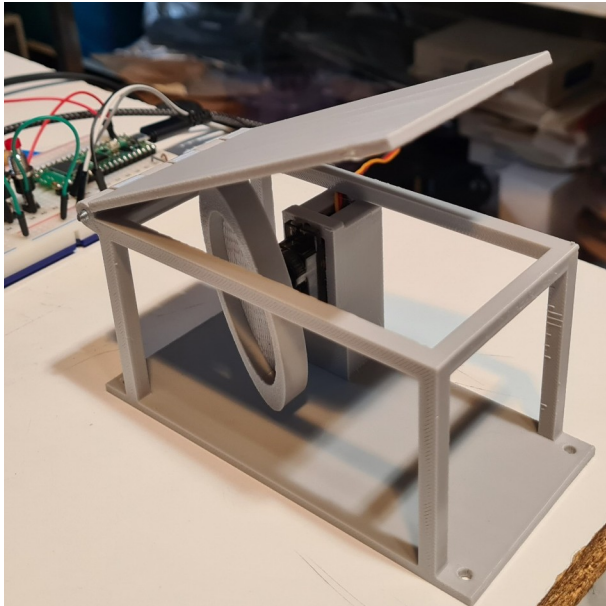
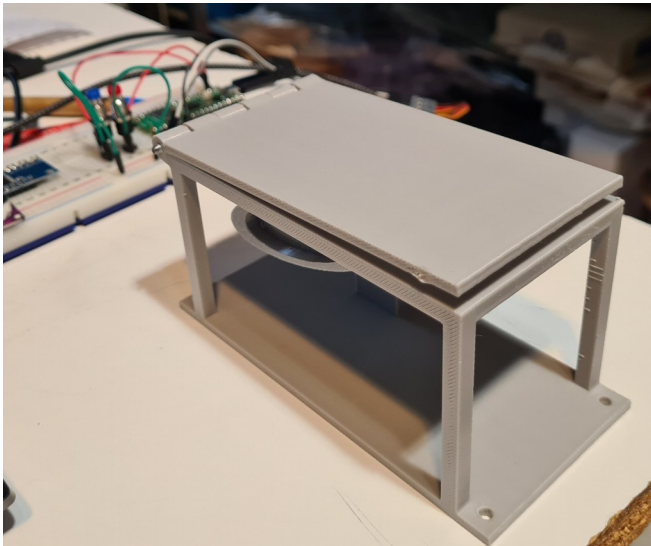
A15E13 Application 1



260127 serre servo simple.py ×

```
1 # Pilotage d'un servo-moteur SG90 pour lever le capot de la serre
2 # on va donner une impulsion au servo-moteur et il va tourner d'un certain angle
3 # l'asservissement de la position est géré par l'électronique du servo moteur
4 # on va le commander avec les boutons poussoirs ouvrir et fermer
5 # l'angle est déterminé à l'avance, une valeur pour ouvert et une valeur pour fermé
6
7
8 #initialisation
9 from machine import Pin, PWM #import des fonctions que l'on a besoin à partir des modules "machine, time,..."
10
11 from time import sleep_ms
12
13 # initialisation des boutons, N° du GPIO, Pin en entrée, Pin mis à 3.3V (tension haute)
14 #le niveau de tension sur l'entrée du GPIO va baisser à 0V lorsque l'on va appuyer sur le BP
15 BP_Ferme = Pin(0, machine.Pin.IN, Pin.PULL_UP) # bouton de gauche
16 BP_Ouvre = Pin(1, machine.Pin.IN, Pin.PULL_UP) # bouton de droite
17
18 servo = PWM(Pin(12))# initialisation de la commande du PWM sur le Pin GPIO17
19
20 servo.freq(50) #PWM à 50 hertz fréquence fixe, soit 20ms de temps de cycle, après il faut faire
21 # varier le rapport cyclique pour changer l'angle du servo
22
23
24
25 while True:
26     if BP_Ouvre.value() == 0:
27
28         Pulse = 8000
29         servo.duty_u16(Pulse)
30
31     if BP_Ferme.value() == 0:
32         Pulse = 5000
33         servo.duty_u16(Pulse)
34
35     sleep_ms(100)
36
37 # si l'on appuie deux fois de suite sur 1 des boutons, la consigne ne change pas donc le servo moteur ne bouge pas
```


A15E14 Application 2



260127 serre servo variable.py ×

```
1 # Pilotage d'un servo-moteur SG90 pour lever le capot de la serre
2 # on va donner une impulsion au servo-moteur et il va tourner d'un certain angle
3 # on va le commander aussi avec les boutons poussoirs ouvrir et fermer
4 # si on relache le PB, le servo s'arrêtera en position intermédiaire
5
6 #initialisation
7 from machine import Pin, PWM #import des fonctions que l'on a besoin à partir des modules "machine, time,..."
8
9 from time import sleep_ms
10
11 # initialisation des boutons, N° du GPIO, Pin en entrée, Pin mis à 3.3V (tension haute)
12 #le niveau de tension sur l'entrée du GPIO va baisser à 0V lorsque l'on va appuyer sur le BP
13 BP_Ferme = Pin(0, machine.Pin.IN, Pin.PULL_UP) # bouton de gauche
14 BP_Ouvre = Pin(1, machine.Pin.IN, Pin.PULL_UP) # bouton de droite
15
16 servo = PWM(Pin(12))# initialisation de la commande du PWM sur le Pin GPIO17
17
18 servo.freq(50) #PWM à 50 hertz fréquence fixe, soit 20ms de temps de cycle, après il faut faire
19 # varier le rapport cyclique pour changer l'angle du servo
20
21 # definition de l'increment de pas du servomoteur
22 Increment=100
23
24 # positionnement à 0 capot fermé
25 Pulseinit = 5300 #On va pouvoir aller de "pulse"= 5300 à 8300 représentant un angle de rotation
26 #de 0° à 90° du servo-moteur,
27 servo.duty_u16(Pulseinit) #fait tourner le servo-moteur de l'angle correspondant à "pulseinit"
28 print('capot fermé')
29 Pulse = Pulseinit # "Pulse" va être la variable que l'on va faire bouger pour commander le servo moteur
30 # on l'initialise avec la valeur de "pulseinit" au début
31 sleep_ms(50)
32
33
34 while True :
35     # on ouvre
36     if 5200<=Pulse<= 8300: # arrêt de la commande à 8300
37         if BP_Ouvre.value() == 0: # if BP_Ouvre.value() == 0: # on teste si le niveau
38             #de tension sur le GPIO est à 0V ou pas
39             # si oui, alors on fait ce qu'il y a sur les lignes suivantes
40
41             Pulse = Pulse + Increment # Pulse= Pulse + Increment #on ajoute la valeur de la variable Increment
42             # à l'angle précédent
43             # il peut monter à 8200+100, il faut pouvoir redescendre
44             # donc 8400 au "if" ci dessous
45             print("Pulse = ", Pulse, " on ouvre")
46             servo.duty_u16(Pulse)
47
48     # on ferme
49     if 5300<= Pulse<= 8400: # arrêt de la commande à 5300
50         if BP_Ferme.value() == 0: # if BP_Ferme.value() == 0: # on teste si le niveau de tension
51             #sur le GPIO est à 0V ou pas
52             # si oui, alors on fait ce qu'il y a sur les lignes suivantes
53             # il peut descendre à 5300 -100, il faut pouvoir remonter
54             #donc 5200 au "if" ci dessus
55             print("Pulse = ", Pulse, " on ferme")
56             servo.duty_u16(Pulse)
57             sleep_ms(100)
58
59     # en dehors de la plage 5200-8400, on ne fait rien
```

Propositions de tests pour aller plus loin :

Ecrire un test avec les fonctionnalités suivantes :

- le PIR détecte une présence, la fenêtre de la serre s'ouvre
- le PIT ne détecte personne, la fenêtre de la serre se ferme

Ecrire un test, qui ouvre la fenetre en fonction de la température intérieure

Ecrire un test qui indique l'angle d'ouverture du servo sur les leds

Prog 1

```
1 from machine import Pin, PWM
2 from time import sleep
3
4 servo = PWM(Pin(12)) # fil jaune sur GPIO 12
5 servo.freq(50)
6
7 while True:
8     servo.duty_u16(2000) # gauche - angle 0 degrés
9     sleep(1)
10
11     servo.duty_u16(5000) # milieu - angle 90 degrés
12     sleep(1)
13
14     servo.duty_u16(8000) # droite - angle 180 degrés
15     sleep(1)
```

Prog 2

```
1 from machine import Pin, PWM
2 import time
3
4 #Declaration du bouton
5 bouton = Pin(0, machine.Pin.IN, Pin.PULL_UP)
6
7 #Declaration du servo
8 servo = PWM(Pin(12))
9 servo.freq(50)
10 servo.duty_u16(2000)
11
12 #Declaration d'une variable compteur
13 compteur = 2000
14
15 while True:
16     #On regarde l'etat du bouton
17     if bouton.value() == 0:
18         compteur = compteur + 1000
19         print("Compteur = ",compteur)
20
21     #On verifie et on corrige si besoin
22     #les valeurs du compteur
23     if (compteur > 8000):
24         compteur = 8000
25
26     #On envoie la valeur au servo
27     servo.duty_u16(compteur)
28
29     #On attends un peu (pour éviter les rebonds du BP)
30     time.sleep(0.3)
31
```

Prog 3

```
1 from machine import Pin, PWM
2 import time
3
4 #Declaration du bouton
5 bouton = Pin(0, machine.Pin.IN, Pin.PULL_UP)
6 bouton2 = Pin(1, machine.Pin.IN, Pin.PULL_UP)
7
8 #Declaration du servo
9 servo = PWM(Pin(12))
10 servo.freq(50)
11 servo.duty_u16(2000)
12
13 #Declaration d'une variable compteur
14 compteur = 2000
15
16 while True:
17     #On regarde l'etat du bouton 1
18     if bouton.value() == 0:
19         compteur = compteur + 1000
20         print("Compteur = ",compteur)
21
22     #On regarde l'etat du bouton 2
23     if bouton2.value() == 0:
24         compteur = compteur - 1000
25         print("Compteur = ",compteur)
26
27     #On verifie et on corrige si besoin
28     #les valeurs du compteur
29     if (compteur > 8000):
30         compteur = 8000
31     if (compteur < 2000):
32         compteur = 2000
33
34     #On envoie la valeur au servo
35     servo.duty_u16(compteur)
36
37     #On attends un peu (pour éviter les rebonds du BP)
38     time.sleep(0.3)
39
```